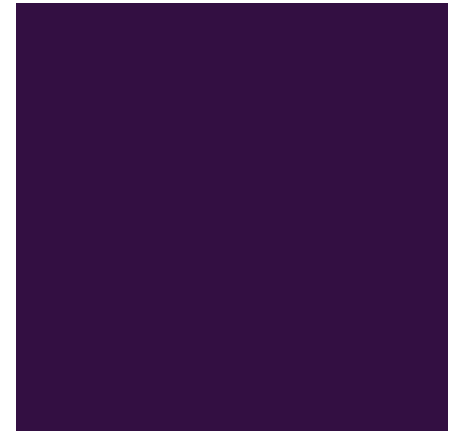
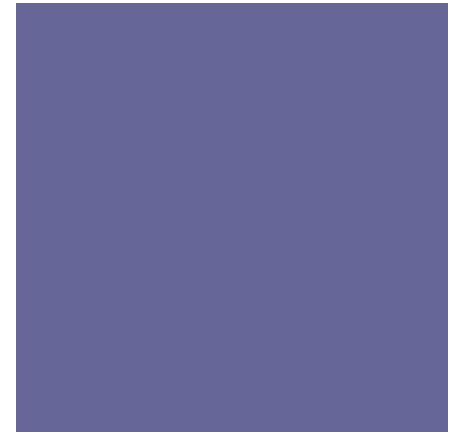




Kiến trúc máy tính



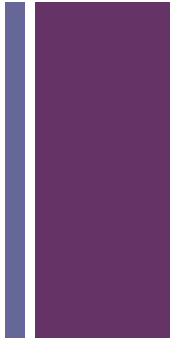
+

Chương 11

Tập lệnh: Các chế độ định địa chỉ
và định dạng lệnh



Chương 11. Chế độ định địa chỉ và định dạng lệnh

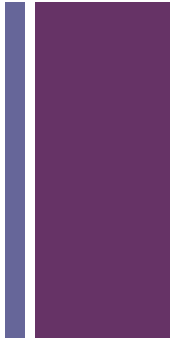


- 11.1 Các chế độ địa chỉ
- 11.2 Các chế độ địa chỉ của x86 và ARM
- 11.3 Định dạng lệnh
- 11.4 Định dạng lệnh của x86 và ARM
- 11.5 Hợp ngữ (Assembly Language)



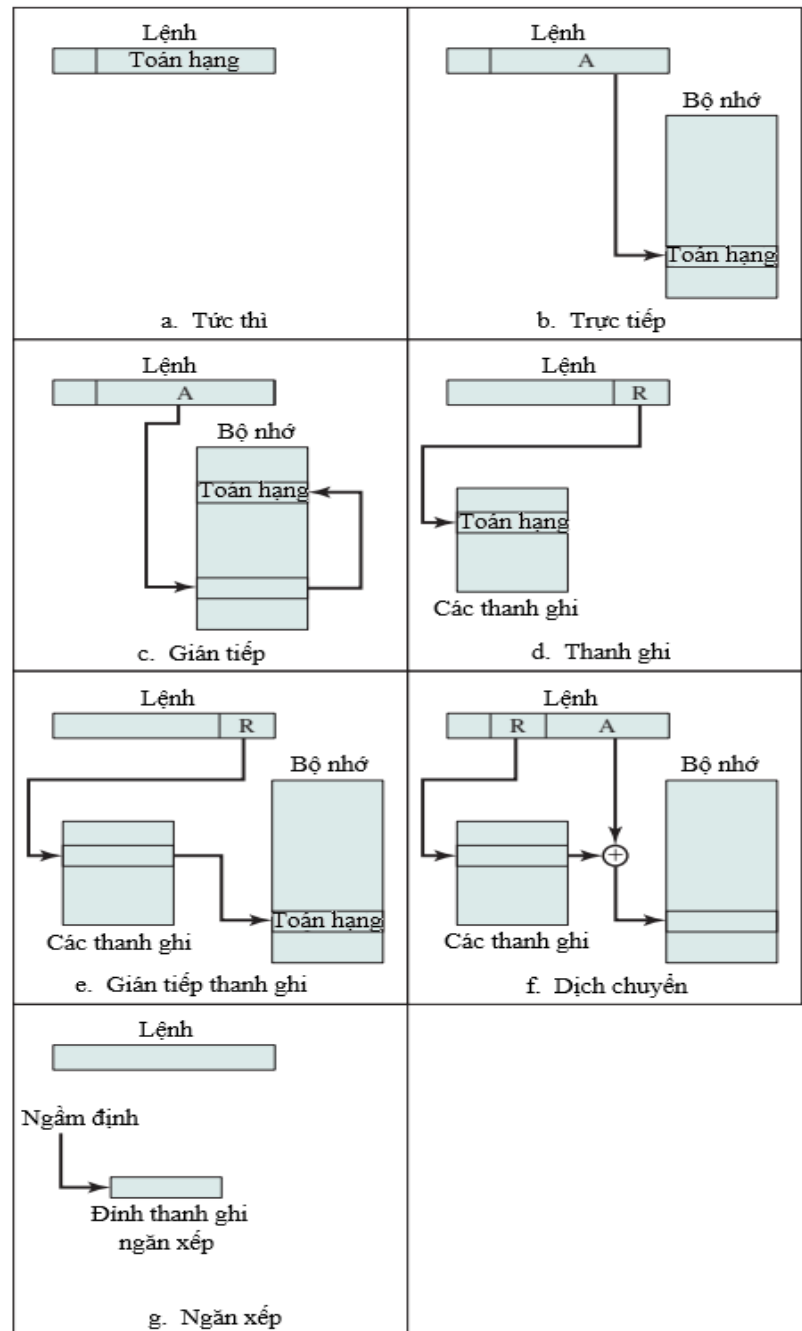
11.1 Các chế độ định địa chỉ

- a. Tức thì
- b. Trực tiếp
- c. Gián tiếp
- d. Thanh ghi
- e. Gián tiếp thanh ghi
- f. Dịch chuyển
- g. Ngăn xếp





Các chế độ định địa chỉ





a. Định địa chỉ tức thì

- Dạng đơn giản nhất của định địa chỉ

Lệnh

- Toán hạng = A



- Chế độ này có thể được sử dụng để định nghĩa và sử dụng các hằng số và thiết lập các giá trị ban đầu của biến

- Các số thường được lưu trữ dưới dạng số bù hai
- Bit ngoài cùng bên trái của trường toán hạng được sử dụng như bit dấu

- Ưu điểm: **ADD 3: acc+3 --> acc**

- Chỉ cần truy xuất bộ nhớ một lần (để lấy lệnh), do vậy tiết kiệm một chu kỳ cache hoặc bộ nhớ trong chu kỳ lệnh.

- Nhược điểm:

- Kích thước của số bị giới hạn bởi kích thước của trường địa chỉ vì thông thường kích thước của trường này nhỏ hơn kích thước từ



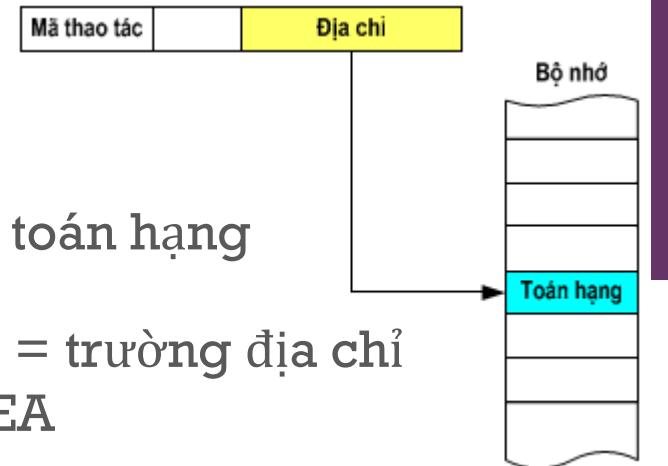
Ví dụ:

ADD R1, 5 ; R1 ← R1 + 5

Trong đó, 5 là một toán hạng nguồn được tham chiếu trực tiếp trong câu lệnh: địa chỉ tức thì

+

b. Định địa chỉ trực tiếp



- Trường địa chỉ chứa địa chỉ hiệu dụng của toán hạng
- Địa chỉ hiệu dụng (Effective address (EA)) = trường địa chỉ (Address field (A)) 0011 0001 1010 0000: EA
- Phổ biến trong các thế hệ máy tính trước đây
- Tham chiếu bộ nhớ **một lần** để lấy dữ liệu toán hạng
- Hạn chế: chỉ cung cấp một không gian địa chỉ hạn chế

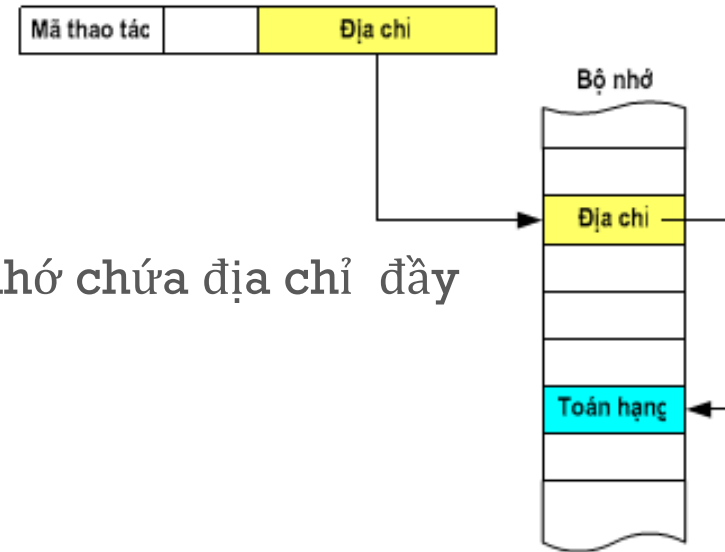
Ví dụ:

ADD R1, A ; $R1 \leftarrow R1 + (A)$

Trong đó, A là địa chỉ một vị trí bộ nhớ, (A) là nội dung của vị trí đó. CPU tham chiếu đến toán hạng có địa chỉ A trong bộ nhớ.

+

c. Định địa chỉ gián tiếp

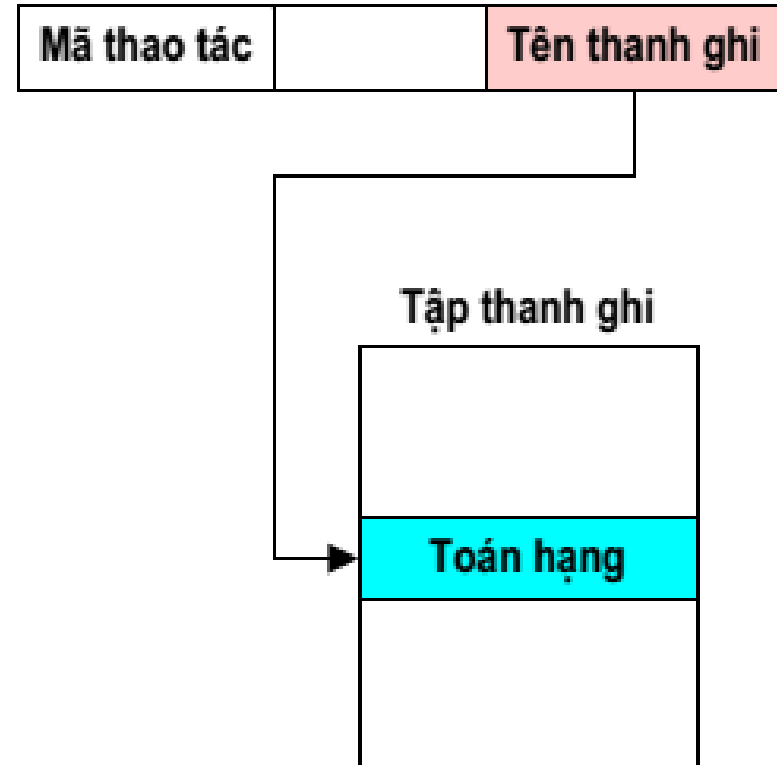


- Tham chiếu đến địa chỉ của một từ trong bộ nhớ chứa địa chỉ đầy đủ của toán hạng
- $EA = (A)$
 - Dấu ngoặc đơn được hiểu như là *nội dung của*
- Ưu điểm:
 - Với một từ có kích thước N cho phép một không gian địa chỉ là 2^N
- Nhược điểm:
 - Thực thi câu lệnh đòi hỏi hai lần tham chiếu bộ nhớ để truy xuất toán hạng
 - Một để lấy ra địa chỉ, hai là để lấy ra giá trị của nó
- Một biến thể hiếm gặp của địa chỉ gián tiếp là địa chỉ gián tiếp nhiều cấp hoặc nhiều tầng
 - $EA = (\dots (A) \dots)$
 - Nhược điểm là cần ba hoặc nhiều hơn tham chiếu bộ nhớ để truy xuất toán hạng



d. Định địa chỉ thanh ghi

- Trường địa chỉ dùng để tham chiếu thanh ghi chứ không phải địa chỉ bộ nhớ chính
- $EA = R$
- Ưu điểm:
 - Chỉ cần một trường địa chỉ nhỏ trong lệnh (do số lượng thanh ghi ít)
 - Không cần tham chiếu bộ nhớ (tốn nhiều thời gian)
- Nhược điểm:
 - Không gian địa chỉ giới hạn



Ví dụ:

ADD R1, A ; $R1 \leftarrow R1 + (A)$

Trong đó, R1 là một thanh ghi trong bộ xử lý, câu lệnh trên tham chiếu giá trị (A) trong bộ nhớ cộng với giá trị lưu trữ trong thanh ghi R1, kết quả được ghi vào R1.

❖ Ví dụ:

❖ MOV BX, DX ; Copy nội dung DX vào BX

❖ MOV AL, BL ; Copy nội dung BL vào AL

❖ MOV AL, BX ; không hợp lệ vì các thanh ghi có kích thước khác nhau

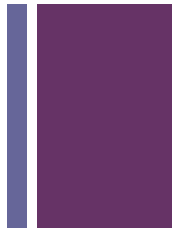
❖ MOV ES, DS ; không hợp lệ (segment to segment)

❖ MOV CS, AX ; không hợp lệ vì CS không được dùng làm thanh ghi đích

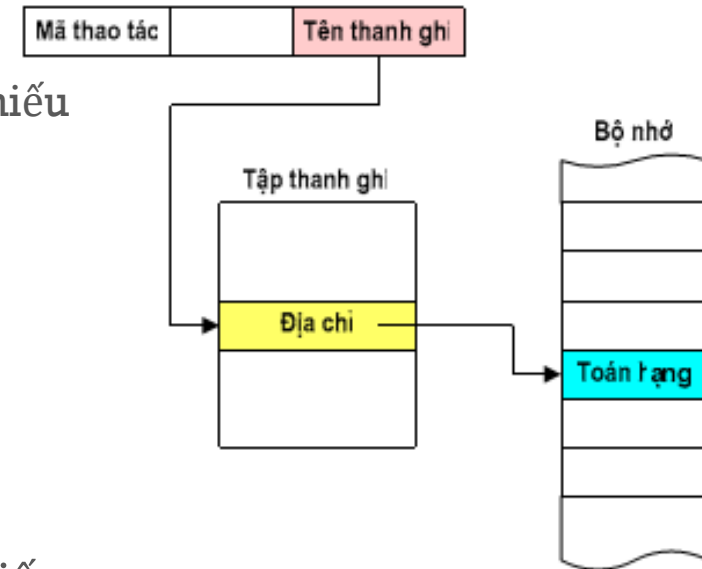
❖ ADD AL, DL ; Cộng nội dung AL



e. Định địa chỉ gián tiếp thanh ghi



- Tương tự như địa chỉ gián tiếp
 - Sự khác biệt duy nhất là trường địa chỉ tham chiếu đến thanh ghi
- $EA = (R)$
- Không gian địa chỉ lớn hơn (trường địa chỉ tham chiếu đến vị trí chứa địa chỉ có độ dài bằng một từ)
- Tham chiếu bộ nhớ ít hơn định địa chỉ gián tiếp



❖ Ví dụ:

- ❖ `MOV AL, [BX]` ; Copy nội dung ô nhớ có địa chỉ DS:BX vào AL
- ❖ `MOV [SI], CL` ; Copy nội dung của CL vào ô nhớ có địa chỉ DS:SI
- ❖ `MOV [DI], AX` ; copy nội dung của AX vào 2 ô nhớ liên tiếp DS: DI và DS: (DI +1)

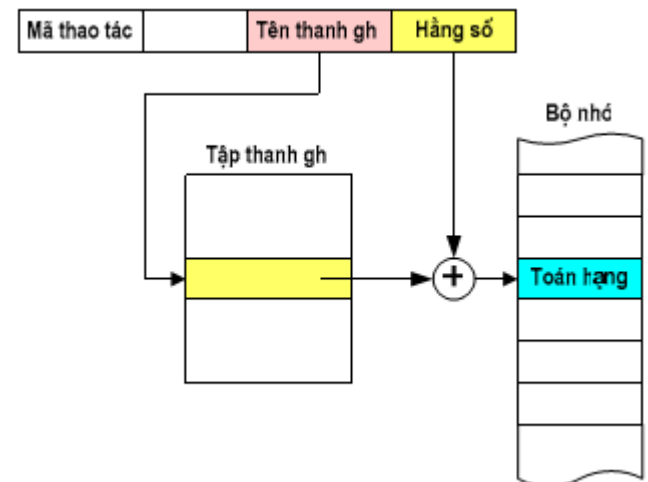


f. Định địa chỉ dịch chuyển - Displacement Addressing

- Kết hợp chế độ định địa chỉ trực tiếp và định địa chỉ trực tiếp thanh ghi
- $EA = A + (R)$
- Yêu cầu lệnh phải có hai trường địa chỉ, ít nhất một trong hai phải có giá trị cụ thể
 - Một giá trị trong một trường địa chỉ (giá trị = A) được sử dụng trực tiếp
 - Một trường địa chỉ khác tham chiếu đến thanh ghi trong đó nội dung được cộng với A để tạo ra địa chỉ hiệu dụng

- Hầu hết sử dụng:

- Định địa chỉ tương đối
- Định địa chỉ thanh ghi cơ sở
- Định địa chỉ chỉ mục



+ Ví dụ

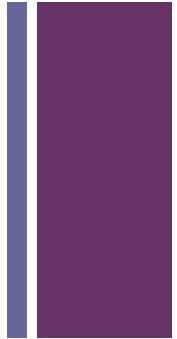
- ❖ **MOV AL, [BP]+5** ; copy nội dung của ô nhớ SS:BP+5 vào thanh ghi AL

+ Định địa chỉ tương đối

- Thanh ghi được tham chiếu ngầm là thanh ghi PC (program counter)
 - Địa chỉ lệnh tiếp theo được cộng vào trường địa chỉ để tạo ra EA
 - Thông thường trường địa chỉ được coi là dữ liệu dạng số bù 2 của hoạt động này
 - Do đó, địa chỉ hiệu dụng là quan hệ dịch chuyển so với địa chỉ của lệnh
- Khai thác tính cục bộ của bộ nhớ
- Lưu các bit địa chỉ trong lệnh nếu hầu hết tham chiếu bộ nhớ tương đối gần lệnh đang được thực thi



Định địa chỉ thanh ghi cơ sở



- Thanh ghi được tham chiếu chứa một địa chỉ bộ nhớ chính và trường địa chỉ chứa một giá trị dịch chuyển so với địa chỉ này
- Tham chiếu thanh ghi có thể rõ ràng hoặc ngầm định
- Khai thác tính cục bộ của tham chiếu bộ nhớ
- Phương tiện hữu ích để thực hiện phân đoạn
- Trong một số trường hợp, một thanh ghi cơ sở duy nhất được sử dụng và được ngầm định
- Các trường hợp khác, người lập trình có thể chọn một thanh ghi để lưu địa chỉ cơ sở của một đoạn và lệnh phải tham chiếu nó một cách rõ ràng

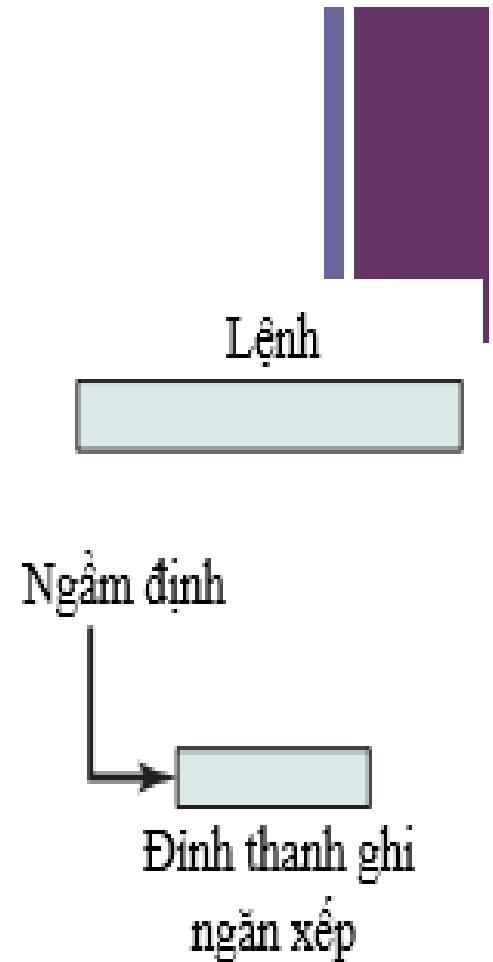


Định địa chỉ chỉ số - Indexed Addressing

- Trường địa chỉ tham chiếu địa chỉ bộ nhớ chính và thanh ghi được tham chiếu chứa giá trị dịch chuyển dương từ địa chỉ này
- Phương pháp tính toán EA giống như với định địa chỉ thanh ghi cơ sở
- Một ứng dụng quan trọng: cung cấp một cơ chế hiệu quả để thực hiện các hoạt động lặp
- Autoindexing
 - Tự động tăng hoặc giảm thanh ghi chỉ mục sau mỗi tham chiếu đến nó
 - $EA = A + (R)$
 - $(R) \square (R) + 1$
- Postindexing
 - Indexing is performed after the indirection
 - $EA = (A) + (R)$
- Preindexing
 - Indexing is performed before the indirection
 - $EA = (A + (R))$

+ Định địa chỉ ngăn xếp

- Một ngăn xếp là một mảng liên tiếp các ô nhớ: danh sách dạng vào trước ra sau
- Lệnh có chế độ địa chỉ này là các lệnh thực hiện trực tiếp với đỉnh ngăn xếp → sử dụng thanh ghi ngàm đỉnh là thanh ghi SP
- Thanh ghi SP (stack pointer – con trỏ ngăn xếp) chứa địa chỉ đỉnh ngăn xếp (trỏ vào đỉnh ngăn xếp)
 - Do đó, chế độ địa chỉ ngăn xếp thực chất là chế độ địa chỉ gián tiếp thanh ghi
- Các lệnh máy không cần tham chiếu bộ nhớ





Các chế độ định địa chỉ cơ bản

Chế độ	Thuật toán	Ưu điểm	Nhược điểm
Tức thì	Toán hạng = A	Không cần tham chiếu bộ nhớ	Hạn chế về giá trị của toán hạng
Trực tiếp	$EA = A$	Đơn giản	Không gian địa chỉ hạn chế
Gián tiếp	$EA = (A)$	Không gian địa chỉ lớn	Tham chiếu bộ nhớ nhiều lần
Thanh ghi	$EA = R$	Không cần tham chiếu bộ nhớ	Không gian địa chỉ hạn chế
Gián tiếp thanh ghi	$EA = A + (R)$	Linh hoạt	Phức tạp
Ngăn xếp	$EA = \text{đỉnh ngăn xếp}$	Không cần tham chiếu bộ nhớ	Khả năng ứng dụng ít



Ví dụ 1

Câu lệnh LOAD: nạp dữ liệu vào thanh ghi ngầm định AC.
Xác định giá trị của AC trong các trường hợp sau. Tính số lần truy xuất bộ nhớ của các lệnh trên

- a. LOAD 1 20 AC=20 ■ Trong đó:
- b. LOAD 2 20 AC=40 ■ 1: chế độ tức thì
- c. LOAD 3 20 vào M[20]=40 ■ 2: chế độ trực tiếp
- 3: chế độ gián tiếp

- d. 40: địa chỉ toán hạng -->
Ac=60

- e. LOAD 1 30

- f. LOAD 2 30

- g. LOAD 3 30

Nội dung các ngăn nhớ	
Địa chỉ	Dữ liệu
20	40
30	50
40	60
50	70



Ví dụ 2

Trường địa chỉ của một câu lệnh là 14 (hệ thập phân). Toán hạng của lệnh nằm ở đâu trong các trường hợp sau

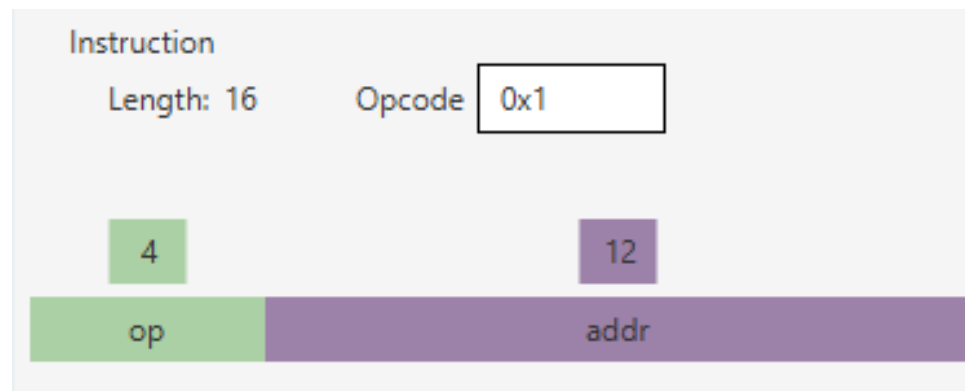
- a) Chế độ địa chỉ tức thì
- b) Chế độ địa chỉ trực tiếp
- c) Chế độ địa chỉ gián tiếp
- d) Chế độ địa chỉ thanh ghi
- e) Chế độ địa chỉ gián tiếp thanh ghi



11.3 Định dạng lệnh

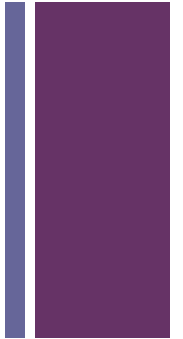
- Định nghĩa cách bố trí các trường bên trong lệnh
- Gồm có các thông tin sau cần được thể hiện:
 - Opcode: mã lệnh
 - Chế độ địa chỉ: ngầm định hoặc cần được chỉ rõ, cho biết chế độ định địa chỉ cho mỗi toán hạng
 - Địa chỉ các toán hạng: ngầm định hoặc được chỉ rõ
- Với mỗi tập lệnh, có thể có nhiều định dạng được sử dụng
- Ví dụ: máy giả thuyết CPUSIM

read	0011 0000 0000 0000
jmpn Done	1011 0000 0000 1010





Kích thước từ lệnh



- Phụ thuộc vào:
 - Kích thước bộ nhớ: vd: cho phép quản lý BN dung lượng bao nhiêu → số bit trường địa chỉ
 - Tổ chức bộ nhớ: kích thước ngăn nhớ, kích thước từ
 - Cấu trúc bus: số đường địa chỉ, số đường dữ liệu → đơn vị truyền
 - Sự phức tạp của bộ vi xử lý: lệnh càng nhiều địa chỉ (kích thước từ lệnh càng lớn) → VXL càng phức tạp
 - Tốc độ bộ vi xử lý
- Kích thước từ lệnh thường bằng hoặc là bội của đơn vị truyền
 - VD: bus dữ liệu có kích thước 16b, kích thước từ lệnh có thể là 16b hoặc 32b tùy kiến trúc



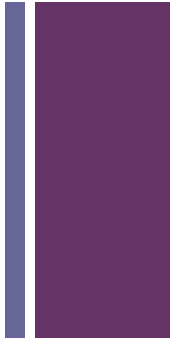
Phân bổ bit

- Việc phân bổ các bit (phân chia các trường) trong một từ lệnh phụ thuộc vào các yếu tố sau:
 - Số lượng của các chế độ định địa chỉ: một lệnh có thể có một hoặc nhiều chế độ địa chỉ, các chế độ này có thể ngầm định hoặc
 - Số lượng các toán hạng
 - Thanh ghi so sánh với bộ nhớ
 - Số lượng tập thanh ghi
 - Dải địa chỉ
 - Địa chỉ chi tiết

Số bit của trường mã lệnh (opcode) quyết định số lệnh tối đa của tập lệnh (quyết định số hoạt động tối đa mà vxl có thể thực hiện được)

CPUSIM – 16b: 4b mã lệnh + 12b địa chỉ

--> 4b mã lệnh: tối đa 16 lệnh = 2^4



Định dạng lệnh PDP-8

Các lệnh tham chiếu bộ nhớ

Opcode		D/I	Z/C	Dịch chuyển						
0	2	3	4	5						11

Các lệnh vào/ra

1	1	0	Thiết bị					Opcode		
0	1	2	3				8	9		11

Các vi lệnh nhóm 1

Các lệnh tham chiếu thanh ghi

1	1	1	0	CLA	CLL	CMA	CML	RAR	RAL	BSW	IAC
0	1	2	3	4	5	6	7	8	9	10	11

Các vi lệnh nhóm 2

1	1	1	0	CLA	SMA	SZA	SNL	RSS	OSR	HLT	0
0	1	2	3	4	5	6	7	8	9	10	11

Các vi lệnh nhóm 3

1	1	1	0	CLA	MQA	0	MLQ	0	0	0	1
0	1	2	3	4	5	6	7	8	9	10	11

D/I = Địa chỉ trực tiếp/gián tiếp

Z/C = Page 0 hoặc page hiện tại

CLA = Xóa thanh ghi AC

CLL = Xóa liên kết

CMA = Thực thi AC

CML = Thực thi liên kết

RAR = Dịch vòng AC sang phải

RAL = Dịch vòng AC sang trái

BSW = Đảo byte

IAC = Tăng AC

SMA = Bỏ qua nếu AC âm

SZA = Bỏ qua nếu AC bằng không

SNL = Bỏ qua nếu liên kết khác không

RSS = Reverse Skip Sense

OSR = Phép Hoặc với thanh ghi Switch

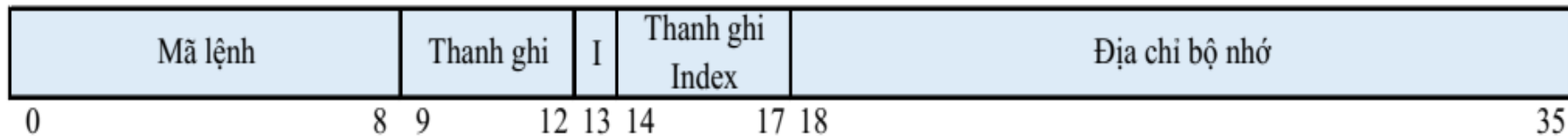
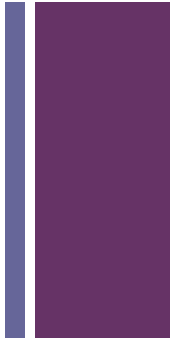
HLT = Dừng

MQA = Nạp số nhân, thương vào AC

MQL = Tải số nhân, thương

+

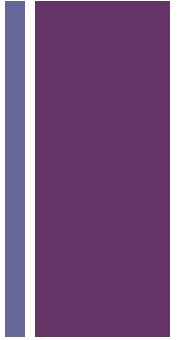
Định dạng lệnh PDP-10



I = bit gián tiếp

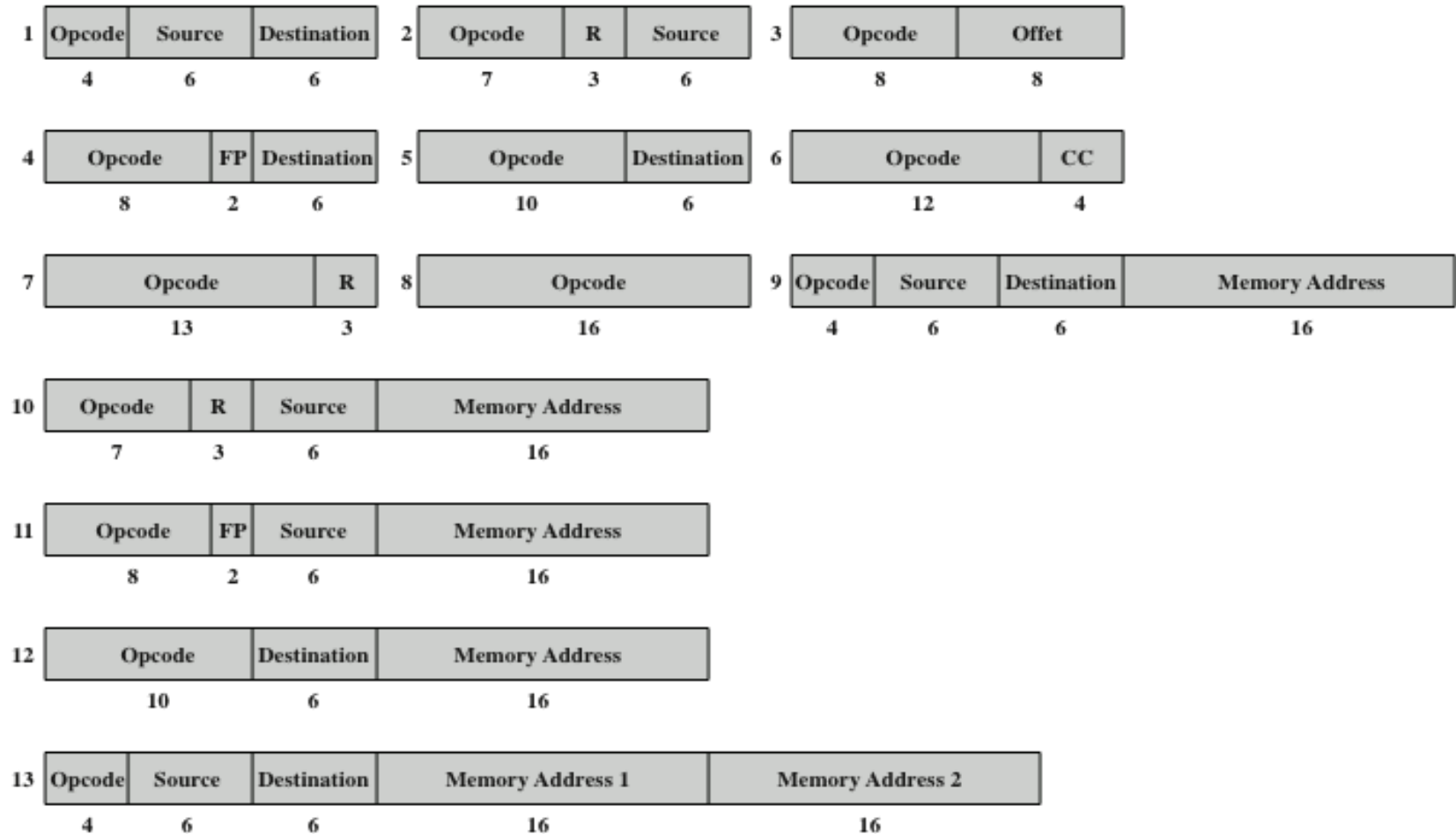


Các lệnh chiều dài thay đổi



- Sự thay đổi có thể được cung cấp hiệu quả và gọn nhẹ
- Tăng sự phức tạp của VXL
- Không loại bỏ mong muốn làm cho tất cả các chiều dài lệnh tương đồng với chiều dài từ
 - Vì VXL không biết chiều dài của lệnh tiếp theo sẽ được truy xuất. Một chiến lược điển hình là truy xuất một số byte hoặc word bằng ít nhất lệnh dài nhất có thể
 - Đôi khi nhiều lệnh được truy xuất

Định dạng lệnh PDP-11



Numbers below fields indicate bit length

Source and Destination each contain a 3-bit addressing mode field and a 3-bit register number

FP indicates one of four floating-point registers

R indicates one of the general-purpose registers

CC is the condition code field

Figure 13.7 Instruction Formats for the PDP-11



Các ví dụ lệnh VAX

Hexadecimal Format	Explanation	Assembler Notation and Description												
8 bits <table><tr><td>0</td><td>5</td></tr></table>	0	5	Opcode for RSB	RSB Return from subroutine										
0	5													
<table><tr><td>D</td><td>4</td></tr><tr><td>5</td><td>9</td></tr></table>	D	4	5	9	Opcode for CLRL Register R9	CLRL R9 Clear register R9								
D	4													
5	9													
<table><tr><td>B</td><td>0</td></tr><tr><td>C</td><td>4</td></tr><tr><td>6</td><td>4</td></tr><tr><td>0</td><td>1</td></tr><tr><td>A</td><td>B</td></tr><tr><td>1</td><td>9</td></tr></table>	B	0	C	4	6	4	0	1	A	B	1	9	Opcode for MOVW Word displacement mode, Register R4 356 in hexadecimal Byte displacement mode, Register R11 25 in hexadecimal	MOVW 356(R4), 25(R11) Move a word from address that is 356 plus contents of R4 to address that is 25 plus contents of R11
B	0													
C	4													
6	4													
0	1													
A	B													
1	9													
<table><tr><td>C</td><td>1</td></tr><tr><td>0</td><td>5</td></tr><tr><td>5</td><td>0</td></tr><tr><td>4</td><td>2</td></tr><tr><td>D</td><td>F</td></tr><tr><td colspan="2"></td></tr></table>	C	1	0	5	5	0	4	2	D	F			Opcode for ADDL3 Short literal 5 Register mode R0 Index prefix R2 Indirect word relative (displacement from PC) Amount of displacement from PC relative to location A	ADDL3 #5, R0, @A[R2] Add 5 to a 32-bit integer in R0 and store the result in location whose address is sum of A and 4 times the contents of R2
C	1													
0	5													
5	0													
4	2													
D	F													

Figure 13.8 Examples of VAX Instructions

Định dạng lệnh x86

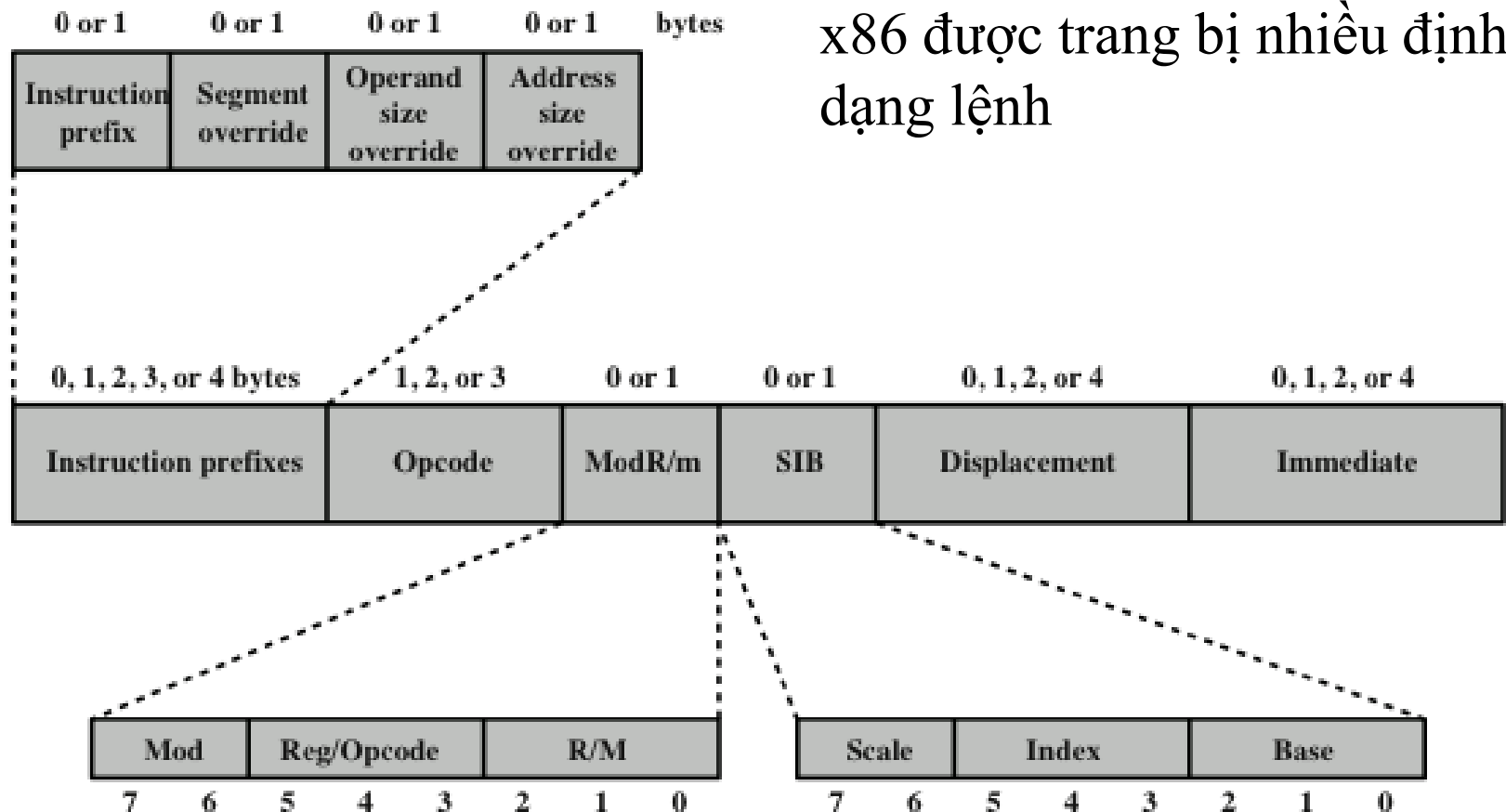


Figure 13.9 x86 Instruction Format

Các định dạng lệnh ARM

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
data processing immediate shift	cond		0			0			0			opcode				S	Rn				Rd				shift amount				shift		0	Rm				
data processing register shift	cond		0			0			0			opcode				S	Rn				Rd				Rs				0	shift		1	Rm			
data processing immediate	cond		0			0			1			opcode				S	Rn				Rd				rotate				immediate							
load/store immediate offset	cond		0			1			0			P	U	B	W	L	Rn				Rd				immediate											
load/store register offset	cond		0			1			1			P	U	B	W	L	Rn				Rd				shift amount				shift		0	Rm				
load/store multiple	cond		1			0			0			P	U	S	W	L	Rn				register list															
branch/branch with link	cond		1			0			1			L	24-bit offset																							

S = For data processing instructions, signifies that the instruction updates the condition codes

S = For load/store multiple instructions, signifies whether instruction execution is restricted to supervisor mode

P, U, W = bits that distinguish among different types of addressing_mode

B = Distinguishes between an unsigned byte (B==1) and a word (B==0) access

L = For load/store instructions, distinguishes between a Load (L==1) and a Store (L==0)

L = For branch instructions, determines whether a return address is stored in the link register

Figure 13.10 ARM Instruction Formats

Ví dụ về sử dụng ARM Immediate Constants (hằng số tức thì)



ror #0 - range 0 through 0x000000FF - step 0x00000001



ror #8 - range 0 through 0xFF000000 - step 0x01000000



ror #30 - range 0 through 0x000003FC - step 0x00000004

Figure 13.11 Examples of Use of ARM Immediate Constants

Tập lệnh Thumb

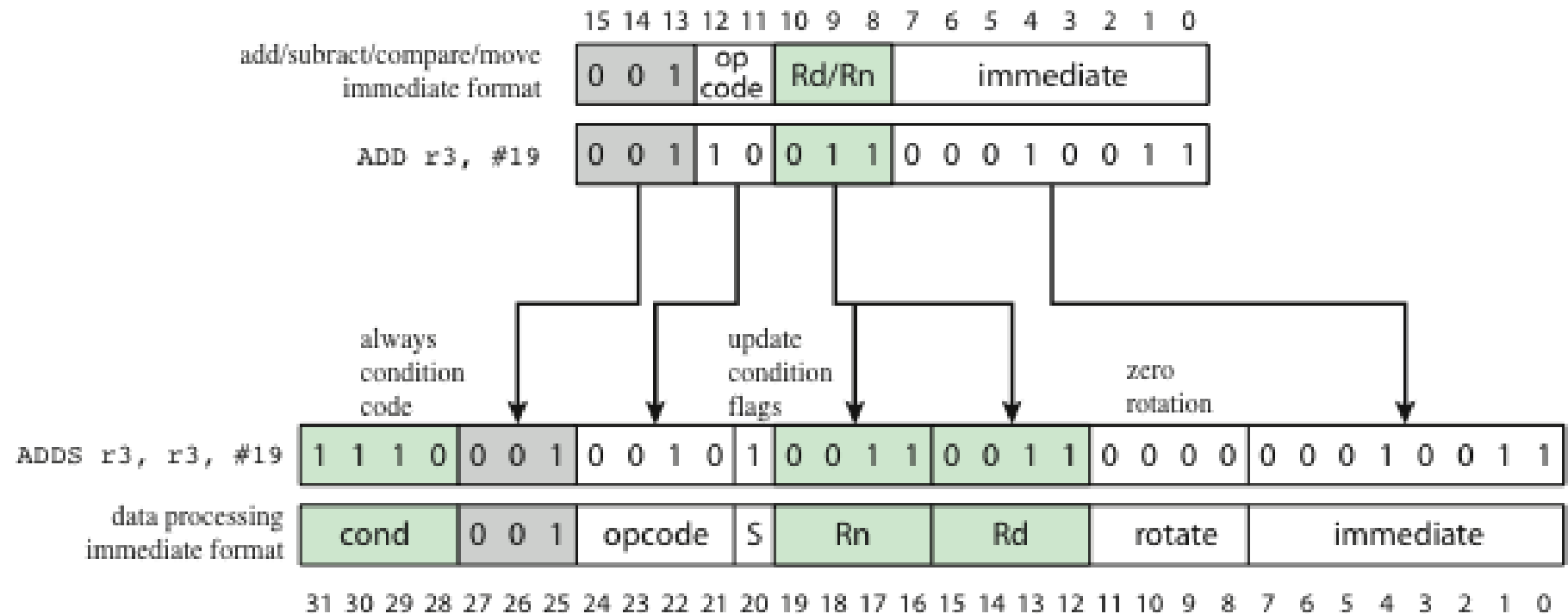


Figure 13.12 Expanding a Thumb ADD Instruction into its ARM Equivalent

Hợp ngữ

Address		Contents		
101	0010	0010	101	2201
102	0001	0010	102	1202
103	0001	0010	103	1203
104	0011	0010	104	3204
201	0000	0000	201	0002
202	0000	0000	202	0003
203	0000	0000	203	0004
204	0000	0000	204	0000

(a) Binary program

Address	Contents
101	2201
102	1202
103	1203
104	3204
201	0002
202	0003
203	0004
204	0000

(b) Hexadecimal program

Address	Instruction	
101	LDA	201
102	ADD	202
103	ADD	203
104	STA	204
201	DAT	2
202	DAT	3
203	DAT	4
204	DAT	0

(c) Symbolic program

Label	Operation	Operand
FORMUL	LDA	I
	ADD	J
	ADD	K
	STA	N
I	DATA	2
J	DATA	3
K	DATA	4
N	DATA	0

(d) Assembly program

Figure 11.13 Computation of the Formula $N = I + J + K$

+ Tổng kết

Chapter 11

- Các chế độ định địa chỉ
 - Định địa chỉ tức thì
 - Định địa chỉ trực tiếp
 - Định địa chỉ gián tiếp
 - Định địa chỉ thanh ghi
 - Định địa chỉ gián tiếp thanh ghi
 - Định địa chỉ dịch chuyển
 - Định địa chỉ ngăn xếp

Tập lệnh: Các chế độ định địa chỉ và định dạng

- Chế độ định địa chỉ x86
- Các chế độ định địa chỉ ARM
- Định dạng lệnh
 - Kích thước lệnh
 - Phân bố các bit
 - Các lệnh có kích thước thay đổi
- Định dạng lệnh X86
- Định dạng lệnh ARM